



COURSE REPORT
INDUSTRIAL INTERNSHIP

**Topic: HumJob: A Local-First Browser Toolkit for the Singing
Voice - Transcription, Analysis, and Ear/Vocal Training**

Supervisor: PhD. Trần Hồng Việt

Author: Vũ Nguyên Đan - 23020351

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

Hà Nội, 2026

Supervisor

ACKNOWLEDGEMENTS

I would like to thank my supervisor for the guidance and feedback received throughout the Industrial Internship course, which shaped both the direction of this work, **HumJob**, and the standard of intellectual honesty I have tried to hold it to.

This report is the work of a single student. Given the breadth of the system and the limited time available, it inevitably contains shortcomings. In particular, the central finding of the transcription evaluation is a sobering one: the aggregate synthetic benchmark the transcriber was tuned against saturated at ceiling, while the system is not yet validated on real hummed input. I have chosen to report that honestly, and to rebuild the evaluation around richer evidence, rather than to present a saturated benchmark as a success. I welcome any comments and corrections from the instructor so that the work may be improved.

ABSTRACT

HumJob is a local-first, browser-based toolkit for the singing voice. It brings together six tools that share one foundation, pitch detection applied to a human voice, and that all run on the user’s own machine: a *melody transcriber* that turns a hummed tune into notes, key, and suggested chords and exports them as MIDI, MusicXML, and engraved sheet music; a *pitch finder* that reports the key, tempo, and Camelot code of any clip, including polyphonic music; a *transposer* that shifts a score to a new key; a *real-time* pitch monitor and vocal trainer; a *sing-along* trainer that scores a sung performance against a reference melody; and an *ear trainer* for interval, chord, scale, and scale-degree recognition. A *practice hub* folds the training history the tools record locally into progress trends. Several features add optional, opt-in language-model assistance, spoken-language coaching, a weekly practice plan, natural-language score edits, a whole-melody rewrite, and reharmonisation, which is the only part of the system that ever leaves the user’s machine.

The technical core of the toolkit, and the subject of this report’s formal evaluation, is the transcriber. It solves monophonic melody transcription from a bare human voice, which is hard because a continuous pitch contour has to be cut into discrete notes with correct pitches, onsets, and durations. HumJob makes this manageable with two deliberate user-interface constraints. The user hums the syllable “da-da-da”, whose consonant gives every note a crisp onset and separates repeated notes of the same pitch; and the user hums to a metronome at a known tempo, which turns rhythm from estimation into snapping onto a known grid. The pipeline is built as a chain of pure functions over three data types, with every tunable parameter centralised, and supports five interchangeable pitch backends (pyin, CREPE, PESTO, FCNF0++, and basic-pitch). A grid-aware segmenter and a consolidation stage that runs for every backend address the dominant failure mode, and a spectral octave-correction stage repairs a systematic pitch-tracker error on legato lines.

The transcriber is evaluated on a synthetic fixture harness that scores note detection (pitch and onset F1) and, separately, rhythm (quantised onset and duration accuracy against an intended grid). On the expressive “realistic” fixtures the note F1 rose from 0.799 to 1.000 across three engineering iterations, and the harness saturated; the rhythm half saturated too once a tempo-refinement stage was added. A single F1 at ceiling cannot separate a better segmenter from a worse one, and it is entirely synthetic, so the evaluation was rebuilt around four kinds of evidence that need no precise singer, since the author is not one and no labelled real-hum dataset exists. These are: diagnostic error-rates over a larger, diversified corpus, which discriminate again and localise the dominant expressive-input failure to merged same-pitch repeats (repeat recall falls to about 0.61); metamorphic correctness properties (transpose, gain, noise, silence-pad, and tempo invariance) that need no reference melody and hold on real audio too; a reconstruction round-trip that scores a real hum against its own resynthesised audio (an unsupervised sanity signal, mean 0.62 over four real takes); and a calibration of the synthetic realism against those real takes, which confirms the assumed vibrato rate and drift are close to real voice. A useful, honest finding is that this pipeline does not over-split under heavy perturbation, as had been expected; its defences hold and it instead degrades by merging or dropping notes. None of the four is a real-world accuracy number, which still requires real ground truth, but together they replace one saturated number with a picture that stays useful. This report documents the whole toolkit, concentrates its evaluation on the transcriber, and gives an honest account of what each kind of evidence can and cannot demonstrate.

Contents

1	Introduction	5
1.1	Motivation and significance	5
1.2	From a transcriber to a toolkit for the voice	5
1.3	Objectives	6
1.4	Research questions	7
2	Background	7
2.1	Melody transcription and the monophonic subproblem	7
2.2	Fundamental-frequency estimation	7
2.3	From a pitch contour to notes	8
2.4	Key finding and rhythm quantisation	8
2.5	Why a metronome and “da-da-da”	8
3	System Design and Methodology	9
3.1	Architecture overview	9
3.2	The transcription pipeline	10
3.3	The note-detection backends	11
3.4	Grid-awareness: segmentation and consolidation	11
3.5	Spectral octave correction	12
3.6	The interactive editor (Manual mode)	12
3.7	The wider toolkit: analysis, training, transposition, and ear training	13
3.8	Optional AI features and the local-first exception	14
3.9	Tempo detection and its known fragility	15
3.10	Technology stack and running the system	15
4	Evaluation and Results	16
4.1	Evaluation methodology	16
4.2	The ground-truth problem	17
4.3	Note-detection results	17
4.4	The rhythm evaluation	18
4.5	Beyond one saturated number: four kinds of evidence	20
4.5.1	Evidence 1: diagnostic error-rates on a diversified corpus	20
4.5.2	Evidence 2: metamorphic correctness properties	21
4.5.3	Evidence 3: a reconstruction round-trip on real hums	21

4.5.4	Evidence 4: calibrating the synthetic realism from real takes	22
4.6	Interpreting the scores	23
4.7	The tempo-detection weakness	24
5	Limitations and Future Work	24
5.1	Evaluation limitations	24
5.2	System deficiencies	25
5.3	Future directions	26
5.4	Towards production readiness	27
6	Conclusion	27
	References	29
	Appendix A: System demonstration	30
A.1	Hub	30
A.2	Transcriber: Auto mode	30
A.3	Transcriber: Manual mode	31
A.4	Pitch Finder	31
A.5	Realtime and vocal trainer	32
A.6	Transposer	32
A.7	Sing-Along and AI coaching	33
A.8	Ear Trainer	33

List of Figures

1	High-level system architecture of HumJob.	9
2	The HumJob transcription pipeline, both note-production paths.	10
3	Rhythm accuracy under timing jitter and under a wrong tempo.	19
4	Hub practice dashboard.	30
5	Transcriber: Auto mode.	31
6	Transcriber: Manual mode.	31
7	Pitch Finder.	32
8	Realtime voice monitor and vocal trainer.	32
9	Transposer: hum mode.	33
10	Sing-Along and AI coaching.	33
11	Ear Trainer.	34

List of Tables

1	The three data types the pipeline is built on.	9
2	The note-detection backends.	11
3	Technology stack.	16
4	Mean note F1 on the realistic fixtures across three iterations. The realistic take is the expressive one (wide vibrato, tremolo, drift, jittered consonant gaps, partial closures).	17
5	Per-fixture note metrics on the realistic take, current state. P, R, F1 are precision, recall, and F1 from mir_eval; ref/est is reference versus detected note counts.	18
6	Rhythm accuracy under two conditions. “on”, “dur”, and “both” are the fraction of notes with the correct onset, correct duration, and both; “err” is the mean onset error in milliseconds. The jitter condition applies human timing jitter (± 30 ms) at the correct tempo; the wrong-tempo condition applies a +5% tempo error to on-grid timing.	19
7	Diagnostic error-rates, corpus means over the sixteen melodies, at three realism profiles. F1 is the mean note F1 for reference. “split” is the signed over/under-split rate (negative is merging or dropping). “octave” is the octave-error rate. “repeat” is the same-pitch repeat recall (higher is better). “off-grid” is the tied-sliver / off-grid rate. No melody collapsed to zero notes at any profile.	20
8	Reconstruction round-trip on the four real recorded takes. “explains” is the summary score, “pitch” is the fraction of jointly-voiced frames within 50 cents, “onset” is the onset-time F1, and “cents” is the median absolute pitch error over jointly-voiced frames. Higher is better except for cents. . .	22
9	Realism quantities measured from the four real takes (medians), next to the value the realistic synthetic profile assumes. The timing-jitter row is the exception: the realistic pitch profile itself keeps timing clean, so the 30 ms shown is the jitter the rhythm test injects (Section 4.4), which is what the measured value is compared against.	23
10	Known deficiencies of the current system and planned work.	25

1 Introduction

1.1 Motivation and significance

A common musical situation has no good tool. A person has a melody in their head, hums it, and wants it written down: as notes on a staff, as a MIDI file they can edit, or as a chord sketch they can play. Turning that hum into notation is the task known as *query-by-humming* transcription, and it is harder than it looks. The human voice produces a single continuous pitch contour that slides, wavers, and drifts; notation requires that contour to be cut into a sequence of discrete notes, each with a definite pitch, a definite start time, and a definite written duration, and it requires a key and a metrical grid within which those durations are meaningful.

Two families of existing tools sidestep parts of the problem rather than solving it. General audio-to-MIDI transcribers are trained on instruments and polyphonic music; on a bare voice they are prone to octave jumps and they do not exploit any of the structure a deliberate hum provides. Real-time pitch monitors and tuners report the instantaneous pitch but never commit to a note segmentation at all. What is missing is a system that is built specifically for a cooperative human who is willing to hum in a way that makes transcription manageable.

HumJob is that system. Its central design decision is to accept two small constraints on how the user hums, and to build the entire pipeline around them:

- The user hums the syllable “**da-da-da**”. The consonant closes the vocal tract briefly at the start of each note, which produces a crisp amplitude onset and, crucially, separates two repeated notes of the same pitch that a legato hum would smear into one.
- The user hums **to a metronome at a known tempo**. A known tempo grid converts rhythm from a hard estimation problem into a snapping problem: note onsets and durations are aligned to the nearest grid position rather than guessed from raw timing.

These constraints are load-bearing. They are what make the problem manageable for a single-student project, and a recurring theme of this report is that the remaining failures of the transcriber are failures to exploit them fully, not failures of the underlying pitch models.

1.2 From a transcriber to a toolkit for the voice

HumJob began as that transcriber, which remains its technical core and the focus of this report’s evaluation. In the course of building it, a family of closely related tools for the singing voice fell out of the same foundation, because every one of them rests on the same operation: estimating the pitch of a human voice, locally and in the browser. The finished system is therefore a toolkit of six tools, presented as tabs in one local-first web application:

- a **Transcriber** that turns a hum into notes, key, and chords, the core pipeline, with an interactive staff editor for correcting its draft and offline or language-model reharmonisation of its chords;
- a **Pitch Finder** that reports the key, tempo, and Camelot code of an uploaded clip, including polyphonic music;
- a **Transposer** that shifts a score, from a file or from the last hum, to a new key;
- a **Realtime** monitor and vocal trainer: a live pitch display, a guitar tuner, and a set of practice drills;
- a **Sing-Along** trainer that plays a reference melody and scores a sung performance note by note, with optional language-model coaching;
- an **Ear Trainer** that plays intervals, chords, scales, and cadences for the user to identify.

A seventh tab, a **practice hub**, is a home dashboard rather than a tool in this sense: it aggregates the training history the other tabs record locally into offline progress trends. Several tabs also expose optional, opt-in language-model assistance, described in Section 3.8.

This report describes the whole toolkit but concentrates its evaluation on the transcriber, because that is where the project’s research contribution and its hardest, most measurable problem lie. The other five tools are engineering deliverables built on the shared foundation; Section 3.7 describes them and Appendix A demonstrates each.

1.3 Objectives

The project pursues the following objectives:

- To design and implement a complete, local-first melody-transcription pipeline that takes a hummed recording and returns notes, key, and suggested chords, exported as MIDI, MusicXML, and engraved sheet music.
- To structure the pipeline as a chain of pure functions over a small, fixed data model, so that every stage is independently testable and every tunable parameter is centralised rather than scattered as magic numbers.
- To support several interchangeable pitch-detection backends behind a single interface, so the note-production model can be chosen or replaced without touching the rest of the pipeline.
- To deliver the pipeline both as a command-line tool and as a browser web application that records audio locally and calls the same code, with an interactive editor for correcting the automatic draft.
- To reuse the same local-first pitch-detection foundation across a family of related tools for the singing voice, analysis, transposition, real-time monitoring, and vocal and ear training, so that the core serves more than one musical task.

- To build an evaluation harness that measures the transcriber honestly, and to characterise clearly what that harness can and cannot demonstrate.

1.4 Research questions

The project is organised around three research questions. They concern the transcriber specifically, since that is where the investigative work lies; the other tools are assessed as functional deliverables (Section 3.7, Appendix A).

- **RQ1 - Do the interface constraints help?** Do the two deliberate constraints (a consonant onset per note, and humming to a known tempo grid) turn melody transcription from an estimation problem into a manageable segmentation and snapping problem?
- **RQ2 - Is this a segmentation problem or a model problem?** When notes come out wrong, does the fault lie chiefly in the segmentation and quantisation of an otherwise-correct pitch contour, rather than in the pitch model itself?
- **RQ3 - How well can the system be evaluated?** How well does the system agree with ground truth, and what are the methodological limits of that measurement given that the project has only synthetic ground truth and no labelled corpus of real hummed melodies?

2 Background

2.1 Melody transcription and the monophonic subproblem

Automatic music transcription in general is the conversion of an audio signal into a symbolic score. The general problem is polyphonic and very hard. HumJob addresses the restricted *monophonic* case: one voice, one note at a time. This restriction removes the need to separate simultaneous pitches, but it does not make the problem trivial, because the difficulty moves from source-separation to *segmentation*: deciding where one note ends and the next begins, and how long each note is in metrical terms.

A monophonic transcription pipeline has three logical stages: estimate a pitch contour over time, cut that contour into discrete note events, and place those events on a key and a rhythmic grid. HumJob follows exactly this shape, and the bulk of its engineering effort is in the second and third stages.

2.2 Fundamental-frequency estimation

The first stage estimates the fundamental frequency (f_0) of the voice, frame by frame. Several families of estimator exist and HumJob supports five of them behind one interface:

- **pYIN** [1] is a classical signal-processing estimator that augments the YIN algorithm with probabilistic thresholds and a Viterbi decoding step. It is fast, dependency-light, and the project default.

- **CREPE** [2] is a convolutional neural network trained to estimate pitch directly from the waveform, and is steady on singing voice.
- **PESTO** [3] is a self-supervised, transposition-equivariant pitch estimator; it is the web application’s default because it is precise on voice while being lighter and faster than CREPE.
- **FCNF0++** [4] is a fully-convolutional pitch and periodicity estimator, a precision peer of PESTO.
- **basic-pitch** [5] is an instrument-trained neural note-transcription model that emits note events directly, bypassing the frame-level contour.

The important background fact for this report concerns a specific failure of Viterbi-decoded trackers. On a continuously voiced legato line, with no silence to reset the decoder, the “stay put” transition prior of a Viterbi decoder can lock a whole note onto its sub-harmonic, reporting it a full octave too low. This is not an audio artifact; it is a decoding artifact, and Section 3.5 describes how HumJob detects and repairs it.

2.3 From a pitch contour to notes

Turning a contour into notes requires three sub-decisions. *Voicing* decides which frames carry pitch at all, gating out breaths and silence. *Segmentation* decides where the boundaries between notes fall, which for a “da-da-da” hum means finding the brief energy dips created by the consonant. *Consolidation* does the opposite where needed: a single held note that vibrato has fragmented into several pieces must be fused back into one. Segmentation and consolidation pull in opposite directions, and balancing them is the core difficulty of the pipeline.

2.4 Key finding and rhythm quantisation

Once notes exist, two musical decisions remain. *Key finding* identifies the scale the melody sits in; HumJob correlates the sung pitch-class distribution against the Krumhansl-Kessler probe-tone profiles for all twenty-four major and minor keys [6]. *Quantisation* snaps note onsets and durations to a metrical grid derived from the known tempo, which is what allows durations to be written as clean note values rather than as awkward fractions. When the tempo is wrong, quantisation produces non-integer durations that notation software renders as strings of tied notes, which is the visible symptom of most rhythm errors.

2.5 Why a metronome and “da-da-da”

The two constraints introduced in Section 1 are the project’s answer to the two hardest sub-problems above. The consonant in “da-da-da” directly supplies the onset cue that segmentation needs, and in particular it is the only reliable way to separate two consecutive notes of the same pitch. The metronome supplies the grid that quantisation snaps to. Neither constraint is a burden for a cooperative user, and together they replace two

estimation problems with two detection problems. The project treats them as fixed assumptions, not as limitations to be optimised away.

3 System Design and Methodology

3.1 Architecture overview

The transcriber is organised as a chain of pure functions over three data types. Figure 1 shows the high-level structure: a core Python package implements the pipeline, a command-line tool calls it directly, and a FastAPI web application records audio in the browser and calls the same pipeline server-side. The same web application also hosts the other five tools of the toolkit as separate browser tabs (Section 3.7); this section describes the transcription pipeline, which is the shared technical core. Everything runs locally, and no audio is ever sent to any third-party service. The exception to local-only operation is a small family of opt-in AI features (Section 3.8): only when the user explicitly presses a button does one of them send a short text or numeric summary, never any audio, to an external language-model API.

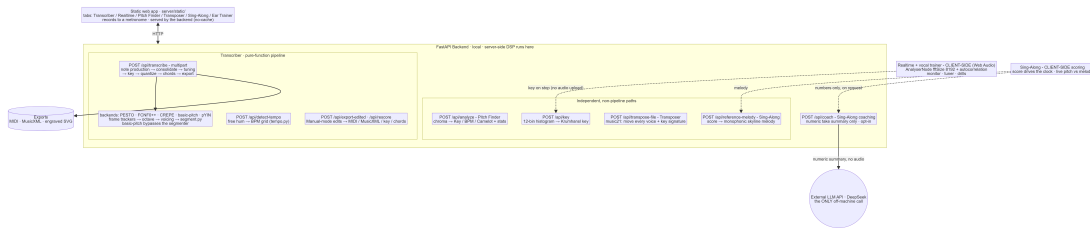


Figure 1: High-level system architecture of HumJob.

The three data types are the contract that every stage shares (Table 1). A **Frame** is one dense analysis frame, roughly one per 11.6 ms hop, carrying the frame time, the estimated f_0 , a voicing confidence, and a short-time energy. A **NoteEvent** is one discrete note after segmentation, carrying its start and end in seconds, its integer pitch, its raw fractional pitch, a cents offset from the nearest semitone, and, once quantised, its grid-aligned position and duration in quarter-note units. A **Score** is the whole transcription, ready to export.

Table 1: The three data types the pipeline is built on.

Type	Role
Frame	One analysis frame (≈ 11.6 ms): frame time, f_0 (NaN if unvoiced), voicing confidence, short-time energy. What the pitch tracker sees.
NoteEvent	One discrete note after segmentation: start/end seconds, integer MIDI pitch, raw fractional pitch, cents offset, and (after quantisation) grid position and duration in quarter-note units.
Score	The whole transcription: the note sequence, the detected key, tuning offset, and chords, ready to export.

A design rule the project holds to is that every tunable parameter lives in a single central parameter object, not as a magic number inside a stage. Changing behaviour means changing a named knob, which keeps the stages readable and the experiments reproducible.

3.2 The transcription pipeline

Figure 2 shows the pipeline stages in order. The chain is:

audio → preprocess → [note production] → consolidate → tuning → key → quantise → chords → export.

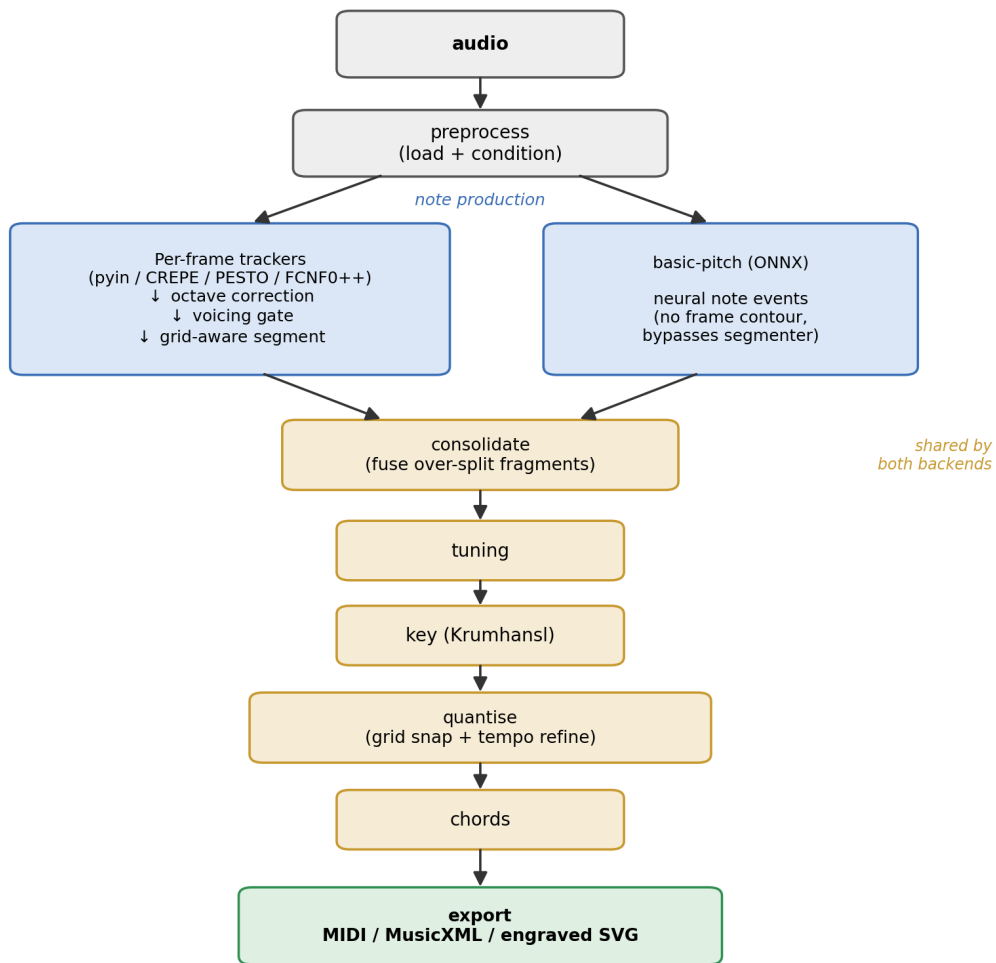


Figure 2: The HumJob transcription pipeline, both note-production paths.

Note production has two paths, selected by a backend parameter. The neural note-event backend (basic-pitch) emits notes directly and has no frame contour. The per-frame trackers (pyin, CREPE, PESTO, FCNF0++) produce a **Frame** contour that then passes through octave correction, a voicing gate, and the grid-aware segmenter to become notes.

Consolidation runs for every backend. It fuses the fragments that a held, vibrato-laden note leaves behind, which is the fix for the “one note becomes many slivers” failure. It is grid-aware: it will not fuse two same-pitch notes across a grid onset, so it never undoes a re-articulation that the segmenter deliberately split on the beat.

The remaining stages apply a global tuning correction, find the key by the Krumhansl-Kessler correlation, quantise onsets and durations onto the tempo grid, suggest chords, and export to MIDI, MusicXML, and engraved sheet music.

3.3 The note-detection backends

The five backends trade precision, speed, and training domain differently. Table 2 summarises them. The default differs by entry point, which is intentional: the library default is `pyin` (fast and dependency-light), the command-line default is `basic-pitch`, and the web application default is `PESTO` (most precise on voice). The four per-frame trackers all feed the same segmenter; only `basic-pitch` bypasses it.

Table 2: The note-detection backends.

Backend	What it is	Best for / notes
<code>pesto</code>	Self-supervised, transposition-equivariant pitch estimator	Most precise on voice; web-app default; lighter and faster than CREPE
<code>fcnf0</code>	FCNF0++, a fully-convolutional f_0 estimator	Precision peer of PESTO; shines on real voice; optional install
<code>crepe</code>	Neural pitch CNN, voice-trained	Steady on humming and singing; uses the accurate full model with Viterbi decoding
<code>basic_pitch</code>	Instrument-trained note-transcription CNN	Instrument clips; can octave-jump on a bare voice; bypasses the segmenter
<code>pyin</code>	Classical signal-processing f_0 plus the project segmenter	Crisp staccato “da-da-da”; library default

3.4 Grid-awareness: segmentation and consolidation

The single most important methodological idea in the pipeline is that segmentation is *grid-aware*. The segmenter is passed the known tempo and a fine short-window energy envelope. A short or soft consonant closure between two same-pitch notes appears as a narrow dip in that fine envelope; a width gate separates such a closure from a wide tremolo swell, and a shallower dip is still accepted when it lands on a beat, because the grid says a note boundary is expected there. This is what separates repeats of the same pitch, and it replaced an earlier coarse valley splitter that smeared over short closures.

Consolidation is the necessary balancing force. Because vibrato and breath can fragment one held note into several pieces, a consolidation stage that runs for every backend fuses

same-pitch fragments back together. Its grid-awareness is the safeguard: it refuses to fuse across a grid onset, so segmentation and consolidation cannot fight each other into an oscillation.

3.5 Spectral octave correction

A distinct stage, inserted between the tracker and the voicing gate for the per-frame backends, repairs the Viterbi subharmonic error described in Section 2. The stage runs for every backend and rests on a spectral signature. A genuine fundamental at frequency f has energy at both its odd harmonics ($f, 3f, 5f$) and its even harmonics ($2f, 4f, 6f$). A subharmonic error, in which the tracker has reported half the true frequency, has energy only at the even multiples, because those are the true fundamental and its harmonics; the odd multiples of the reported frequency have little or no energy. The stage therefore computes, per voiced frame, the salience at the odd and even multiples of the reported f_0 , and doubles the reported frequency only when the odd-harmonic salience collapses relative to the even-harmonic salience. The test is deliberately conservative: it leaves genuine fundamentals alone, including missing-fundamental voices, which retain energy at their odd harmonics. Section 4.3 reports the effect of this stage on the evaluation.

3.6 The interactive editor (Manual mode)

The automatic draft is not always correct, and the weak link is segmentation rather than the pitch contour. The web application therefore provides a Manual mode: an in-browser, staff-based editor for fixing the automatic draft. It is almost entirely client-side. It re-implements the score-building and enharmonic-spelling logic of the export stage in the browser, engraves with a vendored WebAssembly build of the Verovio music-notation engine [9], and offers pure editing operations (change pitch, change duration, merge, split, delete to rest, insert) with undo and redo. A reference pitch strip shows the hummed contour against the chosen notes, so the user can see where the draft diverged from what they sang. The client sends the edited notes back to the server only at the edges, to re-export MIDI and MusicXML and to recompute key and chords.

Manual mode also accepts plain-language edits. An “Ask” box takes an instruction such as “merge notes 3 to 5” or “raise the last two notes an octave”; the common phrasings are matched by an offline grammar that runs with no network access, and anything else is turned into a small edit script by an opt-in language model (Section 3.8). Either way the instruction becomes the same validated script, which a pure interpreter in the browser applies as one undoable step, rejecting the whole script if any note reference is out of range. A separate “Reorganise” button goes the other way: rather than a bounded local edit, it rewrites the whole melody into clean, easy-to-read note values in one undoable step (Section 3.8).

The chords under the draft can be reworked from the result card, independently of the note editing. Clicking a chord offers deterministic alternatives, computed offline: for that measure the system ranks candidate chords, within a chosen style set of triads, diatonic sevenths, and secondary dominants, by how much of the measure’s melody each one covers, with a small complexity penalty so a plain triad wins a tie. A “Reharmonise” control instead asks the opt-in language model for a whole new progression in a requested

style (“jazzier”, “simpler for guitar”); the model proposes one chord per measure from a fixed quality whitelist, and the system validates the proposal, spells and Roman-labels it with music21 [8], scores each bar’s melody fit so a clash is shown rather than hidden, and re-engraves the sheet. Applying either result updates the sheet, the playback, the editor, and the transposer together.

3.7 The wider toolkit: analysis, training, transposition, and ear training

The web application is tabbed. Besides the Transcriber described above, five tabs are separate tools that do not go through the transcription segmenter; together with the transcriber they make up the toolkit introduced in Section 1.2. Each reuses the shared foundation, pitch detection and the Web Audio and music-notation stack, for a different musical task. The four voice-input tools below share one design rule with the transcriber: their pure logic is separated from the DOM so it can be unit-tested, and the microphone is released whenever a tab is left.

- **Pitch Finder** analyses an uploaded clip for key, tempo, and a Camelot code, using a chroma-based method that works on polyphonic audio. It reuses the key-scoring and tempo-folding code but not the monophonic segmenter, which would be wrong for a full song.
- **Realtime** is a client-side live pitch monitor and guitar tuner built on the Web Audio API, with a set of vocal-training drills (a target-note lane, a match game, a scale trainer, vibrato analysis, a vocal range finder, and a per-session progress log). A server round-trip cannot be real-time, so this path runs entirely in the browser and only asks the server for the sung key on stop, from a pitch histogram, with no audio upload. The vocal range finder is the one drill that makes an empirical claim about the singer relative to a population: from the measured maximum phonational range it reports a percentile (*wider than X% of adults*). To keep that claim honest, the model’s mean is taken from a cited study of adult phonational ranges [11], while its standard deviation is a clearly-labelled assumption (about four semitones, in line with voice-range-profile literature) rather than a measured value, and both are printed in the interface.
- **Transposer** shifts a score to a new key. A file mode transposes an uploaded MIDI or MusicXML file server-side with music21 [8], moving every voice and the key signature; a hum mode transposes the last transcription client-side. The file mode also offers Camelot-compatible key presets as one-click shifts.
- **Sing-Along** turns an uploaded MIDI or MusicXML score into a karaoke exercise: the melody is played back as a guide while the user’s voice is tracked live and scored note by note. Because the whole melody is known in advance, the score itself drives the clock; there is no free-timing alignment. The server reduces a possibly polyphonic upload to a single monophonic melody line (its skyline, with ties stripped), and the browser schedules the count-in and the guide notes at absolute audio-clock times and scores the sung pitch against the active note. The in-tune tolerance is selectable (four difficulty levels), the guide volume is adjustable, and the take can be paused and resumed.

- **Ear Trainer** is the mirror of the other tabs: instead of the user making a sound for the system to analyse, the system plays a sound for the user to identify. It is fully client-side, with no microphone and no server. Four sub-modes, selected like the Realtime drills, cover interval, chord-quality, scale-or-mode, and scale-degree recognition; each question plays the same piano samples the other tabs use and offers multiple-choice answers, with three difficulty pools and a per-mode accuracy history kept in the browser. The key sub-mode tests scale-degree recognition after a I-IV-V-I cadence, the trainable relative-pitch skill, rather than absolute-key naming, which would require perfect pitch.

3.8 Optional AI features and the local-first exception

The system exposes five optional, opt-in features backed by an external large-language model, and they are the only places where any data leaves the machine. They share one design contract, so the exception to local-only operation stays small and uniform. Each fires only on an explicit button press; each sends only a compact text or numeric summary, never audio, never a recording, and not even a file name; and each renders the model’s reply as plain text rather than as markup, so untrusted output cannot inject content into the page. The API key is read server-side, per request, from a local, git-ignored configuration file and is never exposed to the browser. All five share a single transport module, and all are off by default and inert until a key is configured, so the rest of the system remains strictly local.

The five features are:

- **Sing-Along coaching.** After a take, the client first computes a deterministic, human-readable analysis of the performance in the browser: the signed pitch bias (consistently sharp or flat), drift from the start of the take to the end, octave slips, accuracy of leaps versus stepwise motion, the weakest register, and the worst individual notes with their bar numbers. This analysis is always shown and needs no network access. Only on request is its numeric summary, with the musical context, sent for spoken-language feedback and practice suggestions.
- **Practice plan.** On the hub, the aggregate progress report, computed locally from the training history, can be sent for a one-week plan whose items name the application’s own drills. The last plan is cached in the browser and keyed by a hash of the report, so opening the hub never issues a request on its own.
- **Natural-language editing.** In Manual mode, the “Ask” box sends a text listing of the melody and the user’s instruction, and the model returns a small structured edit script; the browser validates and applies it. Common instructions are handled entirely offline by a grammar and never reach this path.
- **Whole-melody reorganisation.** Also in Manual mode, a “Reorganise” button sends the current melody’s pitches and beat lengths and receives a cleaned-up rewrite, with durations snapped to simple note values and tied slivers merged; the browser validates it and applies it as one undoable step. Where the Ask box makes a bounded local edit, this rewrites the whole line for readability.

- **Reharmonisation.** On the result card, a requested style and the per-measure pitch-class profile of the melody are sent, and the model proposes a new chord progression, which the system validates, spells, scores, and engraves locally.

The reply requests are handled server-side by synchronous route handlers, which the web framework runs on a worker thread, so a call of up to a minute does not block the event loop. The default model is a reasoning model that spends hidden tokens before its answer, so the structured features (editing, reorganisation, and reharmonisation) request a larger token budget than the prose features and can be pointed at a faster model through configuration. Beyond these five calls, HumJob’s operation is entirely local.

3.9 Tempo detection and its known fragility

Because a wrong tempo is the usual cause of tied-note artifacts, the application offers a “find my tempo” button. Its purpose is narrow: the user hums a few even beats, the system estimates a comfortable tempo, and the user then records the real take to a click at that tempo, so that quantisation lands on whole beats.

The estimator computes an onset-strength envelope with librosa [7], then finds the dominant periodicity of that envelope by autocorrelation, weighted by a prior centred on a comfortable humming tempo, and folds the result into a plausible range by octaves. The important property, and a known weakness examined in Section 4.7, is that this returns a *single global tempo* averaged over the whole clip, tilted toward the prior. It assumes a roughly constant tempo and reasonably clean onsets, and it is fragile when either assumption is violated.

A second, independent safeguard sits in the quantiser itself. Because even a careful hum drifts a few percent off the metronome, the quantiser refines the tempo before snapping: it searches a bounded band around the stated tempo (about $\pm 25\%$) and adopts the beat length that makes the snapped rhythm metrically simplest, leaving a take that is already on the beat untouched. The score is still notated at the tempo the user set; only the grid the onsets snap to follows the performed tempo. This absorbs a moderate tempo error, whether from the user or from the detector above, but not a gross one, which is why the detector’s fragility still matters for large errors (Section 4.7).

3.10 Technology stack and running the system

Table 3 lists the main components. The whole system runs locally. The web application is a FastAPI server that serves a static browser front-end and reloads on source edits; the command-line tool transcribes a file directly.

Table 3: Technology stack.

Layer	Technology
Core pipeline	Python 3.12, NumPy
Pitch backends	pYIN via librosa [7]; torchcrepe [2]; pesto-pitch [3]; penn / FCNF0++ [4]; basic-pitch on ONNX Runtime [5]
Onset / tempo / chroma	librosa [7]
Score model, MIDI, MusicXML	music21 [8]
Engraving	Verovio [9] (server-side, and a vendored WebAssembly build in the browser)
Web backend	FastAPI, Uvicorn (with automatic reload)
Web front-end	Static HTML / CSS / JavaScript, Web Audio API
AI features (optional)	Shared httpx client to an external, OpenAI-compatible language-model API (DeepSeek); powers coaching, the practice plan, natural-language editing, and reharmonisation; off by default, opt-in, text or numeric summaries only
Evaluation	mir_eval [10] for note metrics

4 Evaluation and Results

4.1 Evaluation methodology

This section evaluates the transcriber, the toolkit’s technical core and the only component that estimates a hidden ground truth. The other five tools are functional deliverables rather than objects of measurement: their pure logic (the client score-building and enharmonic spelling, the transposer’s key arithmetic, the sing-along scoring, and the ear-trainer question generation) is covered by unit tests, and Appendix A demonstrates each running. There is no accuracy benchmark for them because they are utilities, not estimators, so the measurements below concern the transcription pipeline alone.

The project’s standard evaluation of the transcriber is a synthetic fixture harness. It renders a set of known melodies as audio, runs the pipeline once per fixture, and compares the output against the known reference. The fixtures cover eight melodies: a C major scale, an A minor scale, an arpeggio, repeated notes, a melody with silence, octave leaps, a Twinkle excerpt, and a mixed-rhythm phrase.

The harness scores two different things, because they measure different failures:

- **Note detection (pitch and onset F1)**, computed with mir_eval [10]. A predicted note matches a reference note when its pitch is correct and its onset is within a tolerance; durations are ignored and no offset tolerance is applied. This is run in two flavours: a *clean* take that the regression gate expects to pass at $F1 = 1.0$, and a *realistic* take with wide vibrato, tremolo, pitch drift, jittered consonant-gap lengths, and partial consonant closures.

- **Rhythm**, added later, because note F1 is rhythm-blind. This scores the quantised onset and duration of each note against an intended grid and is described in Section 4.4.

Both of these are aggregate scores, and both saturated at ceiling during the project (Sections 4.3 and 4.4). Because a saturated number can no longer drive improvement and says nothing about real hums, the evaluation was then widened into four kinds of evidence, presented in Section 4.5, each designed to need no precise performer: diagnostic error-rates on a diversified corpus, metamorphic correctness properties, a reconstruction round-trip on real hums, and a calibration of the synthetic realism against real takes. The saturated F1 results below are the baseline that motivated that widening.

4.2 The ground-truth problem

Before the results, their central limitation must be stated plainly. **All of the ground truth used in this report is synthetic.** The fixtures are melodies the project itself renders to audio, so their intended pitch, onset, and duration are known exactly by construction. This is a genuine strength for regression testing, but it is a genuine weakness for validity: a synthetic hum is more regular than a real one, and a benchmark built only on synthetic hums can be saturated while the system still fails on real input.

The project has no labelled corpus of real hummed melodies. The reason is specific to this task and is the same kind of problem that limits any transcription benchmark: obtaining ground truth for a real hum requires knowing the exact pitch, onset, and duration of every note the person intended, and a person cannot reliably label the pitch of their own hum by ear. As a result, every note-detection and rhythm figure in this report measures agreement with *synthetic* ground truth and must be read as a regression signal, not as a measure of real-world accuracy. Section 5.1 proposes a concrete remedy.

4.3 Note-detection results

On the clean fixtures the pipeline scores a mean note F1 of 1.000. The more informative number is the realistic take, where the segmenter historically struggled. Table 4 records the mean realistic F1 across three engineering iterations. The first raised it from 0.799 to 0.931 by making segmentation and consolidation grid-aware; the second raised it to 1.000 by adding the spectral octave correction of Section 3.5, which fixed the last failing fixture (the octave leaps, previously returned an octave flat).

Table 4: Mean note F1 on the realistic fixtures across three iterations. The realistic take is the expressive one (wide vibrato, tremolo, drift, jittered consonant gaps, partial closures).

Iteration	Change	Mean realistic F1
Baseline	coarse valley splitter	0.799
Grid-aware	grid-aware segment + consolidate	0.931
Octave correction	spectral subharmonic repair	1.000

Table 5 gives the per-fixture note metrics on the realistic take at the current state. Every

fixture scores 1.000 on precision, recall, and F1, and the key is detected correctly in every case. This is the saturation referred to throughout this report: the note-detection half of the synthetic harness no longer discriminates between a better and a worse segmenter, so it can no longer drive improvement on its own.

Table 5: Per-fixture note metrics on the realistic take, current state. P, R, F1 are precision, recall, and F1 from mir_eval; ref/est is reference versus detected note counts.

Fixture	P	R	F1	Key	ref/est
c_major_scale	1.00	1.00	1.00	C major	8/8
a_minor_scale	1.00	1.00	1.00	A minor	8/8
arpeggio	1.00	1.00	1.00	C major	7/7
repeated_notes	1.00	1.00	1.00	C major	5/5
with_silence	1.00	1.00	1.00	C major	5/5
octave_leaps	1.00	1.00	1.00	C major	5/5
twinkle	1.00	1.00	1.00	C major	14/14
mixed_rhythm	1.00	1.00	1.00	C major	5/5
mean			1.000		

4.4 The rhythm evaluation

The note F1 above is rhythm-blind by construction: it ignores durations and accepts any onset within a tolerance, so it says nothing about whether the written rhythm is correct. This matters because the two failures reported most often on real hums are wrong rhythm (durations and onsets on the wrong beats) and over-splitting (one note returned as tied slivers), and neither is visible to note F1. A separate rhythm metric was therefore added. It compares the quantised onset and duration of each note against an *intended grid* read straight from each fixture’s specification, and reports the fraction of notes with the right onset, the right duration, and both. The headline number is “both”: right beat and right written length.

Two conditions were run, chosen to match the two things that actually break real hums. Table 6 gives the per-fixture results.

Table 6: Rhythm accuracy under two conditions. “on”, “dur”, and “both” are the fraction of notes with the correct onset, correct duration, and both; “err” is the mean onset error in milliseconds. The jitter condition applies human timing jitter (± 30 ms) at the correct tempo; the wrong-tempo condition applies a +5% tempo error to on-grid timing.

Fixture	Timing jitter (± 30 ms)				Wrong tempo (+5%)			
	on	dur	both	err	on	dur	both	err
c_major_scale	1.00	1.00	1.00	0	1.00	1.00	1.00	0
a_minor_scale	1.00	1.00	1.00	0	1.00	1.00	1.00	0
arpeggio	1.00	1.00	1.00	0	1.00	1.00	1.00	0
repeated_notes	1.00	1.00	1.00	0	1.00	1.00	1.00	0
with_silence	1.00	0.80	0.80	0	1.00	1.00	1.00	0
octave_leaps	1.00	1.00	1.00	0	1.00	1.00	1.00	0
twinkle	1.00	1.00	1.00	0	1.00	1.00	1.00	0
mixed_rhythm	1.00	1.00	1.00	0	1.00	1.00	1.00	0
mean	0.975				1.000			

Two findings stand out. First, on-grid at the correct tempo the quantiser is perfect (1.000), and under human timing jitter the mean stays at 0.975: every fixture but one scores perfectly, and only the silence fixture loses a written duration, so the quantiser is robust to the timing drift of a real performance. Second, a 5% tempo error is also recovered to 1.000, because the quantiser refines the tempo before snapping (Section 3.9): it searches a bounded band around the stated tempo and adopts the beat length that makes the snapped rhythm metrically simplest, so a hum that drifts a few percent off the metronome is pulled back onto whole beats instead of into the off-beat onsets that notation renders as tied slivers. Within that band the rhythm half of the synthetic harness is therefore saturated as well.

Figure 3 plots the two conditions side by side.

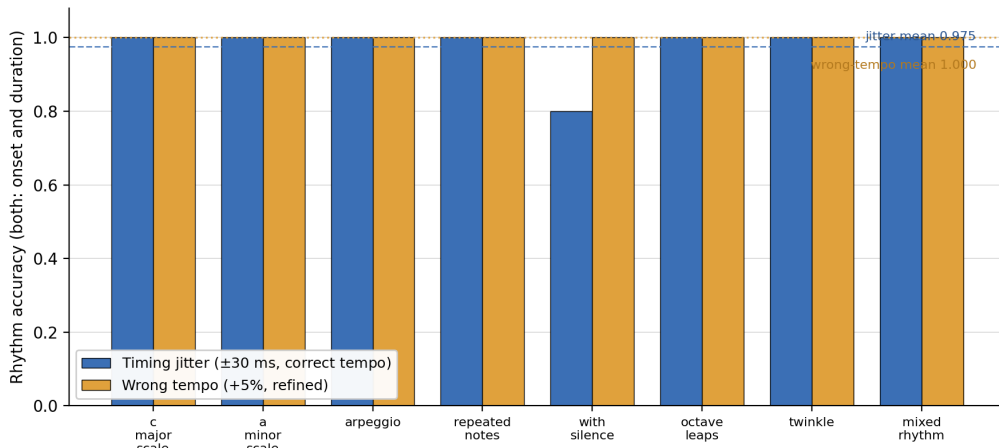


Figure 3: Rhythm accuracy under timing jitter and under a wrong tempo.

4.5 Beyond one saturated number: four kinds of evidence

A single F1 pinned at ceiling cannot separate a better segmenter from a worse one, and it is entirely synthetic, so it proves nothing about real hums. The usual fix, a labelled corpus of real hums, is blocked here for the reason in Section 4.2: the author is not a precise singer and no labelled real-hum dataset exists. The evaluation was therefore widened into four kinds of evidence, each chosen so that it needs no precise performer, and each reported below with an explicit statement of what it does and does not prove. Together they replace one saturated number with a picture that stays useful.

4.5.1 Evidence 1: diagnostic error-rates on a diversified corpus

The first step is to stop hiding failures inside one aggregate. Instead of note F1, the pipeline is scored on a set of diagnostic error-rates that each pick out one failure mode, over a larger, more varied corpus. The corpus has sixteen melodies: the original eight fixtures, three public-domain tunes (Ode to Joy, Mary Had a Little Lamb, Frere Jacques), and five procedurally generated melodies (in-key random walks, an arpeggio, an interval study, and a varied-rhythm line), all rendered with known ground truth. Each melody is rendered at three realism profiles: *clean*, *realistic*, and a new, harder *hard* profile that adds amplitude shimmer, pitch creak, a breathy noise floor, a short room reverberation, and pink microphone noise on top of the realistic vibrato and partial closures.

The error-rates are: the signed over/under-split rate (estimated minus reference note count, over the reference count, so a positive value is over-splitting and a negative value is merging or dropping notes); the octave-error rate (aligned notes off by an exact octave); the same-pitch repeat recall (of the reference’s adjacent same-pitch pairs, the fraction that survive as two distinct notes, which is the “merged repeats” complaint); and the off-grid rate (notes whose quantised onset or duration leaves the simple-note grid, which is the “tied-sliver” complaint). Table 7 gives the corpus means per profile.

Table 7: Diagnostic error-rates, corpus means over the sixteen melodies, at three realism profiles. F1 is the mean note F1 for reference. “split” is the signed over/under-split rate (negative is merging or dropping). “octave” is the octave-error rate. “repeat” is the same-pitch repeat recall (higher is better). “off-grid” is the tied-sliver / off-grid rate. No melody collapsed to zero notes at any profile.

Profile	F1	split	octave	repeat	off-grid
Clean	1.000	+0.00	0.00	1.00	0.00
Realistic	0.979	−0.04	0.00	0.61	0.00
Hard	0.918	−0.04	0.00	0.63	0.00

Three things stand out. First, the corpus discriminates again: the numbers are no longer all 1.000, and they order the three profiles as expected (clean best, hard worst on F1). Second, the most useful single number is the repeat recall, which falls from 1.00 on clean input to about 0.61 on realistic input: on an expressive take the segmenter merges roughly four in ten of the same-pitch repeats it should keep apart. That is the long-suspected “merged repeats” weakness, now measured rather than only reported informally. Third, the octave rate stays at zero (the spectral octave correction holds) and the off-grid rate

stays at zero at the correct tempo (the quantiser keeps notes on simple values).

One expectation was not met, and the report states it plainly. The *hard* profile was designed to provoke *over-splitting* (a held note shattered into slivers), the failure the too-regular synthesiser could not previously show. Direct measurement showed the opposite. This pipeline does not over-split even under heavy perturbation; its median-smoothing and consolidation stages hold, and hard input instead degrades by *merging or dropping* notes. Pushing the synthesiser far enough to fragment a note instead silences it. The signed split rate is therefore mildly negative, never positive. This is a genuine finding about the pipeline’s behaviour, and it corrects the earlier expectation recorded in this report.

What this proves: the pipeline’s behaviour on varied, expressive input is now measured across distinct failure modes, and the corpus can once again tell a better segmenter from a worse one. The dominant expressive-input failure is localised to merged same-pitch repeats, not to octave errors or off-grid rhythm. **What it does not prove:** the ground truth is still synthetic, so the absolute rates depend on how faithful the realism profiles are, which is the subject of Evidence 4.

4.5.2 Evidence 2: metamorphic correctness properties

The second kind of evidence needs no reference melody at all. A metamorphic test asserts a *relation* that must hold between a take and a transformed version of the same take, so the “label” is the transformation, not a known pitch. Five properties are checked: transpose invariance (shifting the pitch by k semitones shifts every output note by k and leaves the count unchanged); gain invariance (scaling the amplitude leaves the transcription identical); additive-noise robustness (adding white noise down to a 30 dB signal-to-noise ratio leaves the notes unchanged); silence-pad invariance (prepending or appending silence leaves the same notes at the same values); and tempo, or time-scale, invariance (performing the same melody at a scaled tempo and reading it at that tempo leaves the note values unchanged). All five hold on the synthetic fixtures. Transpose invariance and gain invariance also hold on the real recorded takes, where they are true regardless of how accurate the hum was.

What this proves: a body of correctness relations the transcriber must obey, verifiable with no precise performer and partly on real audio; a change that broke transpose or tempo handling would be caught here. **What it does not prove:** an invariance can hold while the transcription is wrong in a way the transformation also preserves, so these bound whole classes of error but do not measure absolute accuracy.

4.5.3 Evidence 3: a reconstruction round-trip on real hums

The third kind of evidence is the only one computed on real voice. Because a real hum has no ground truth, the transcription is scored against *its own audio*: the transcribed notes are resynthesised to a plain tone track, and that reconstruction is compared with the original recording. Pitch agreement is the fraction of jointly-voiced frames within 50 cents; onset agreement is an onset-time F1 between the audio’s detected onsets and the transcribed onsets; and a single “explains-audio” summary averages the two. Table 8 gives the result on the four real takes recorded.

Table 8: Reconstruction round-trip on the four real recorded takes. “explains” is the summary score, “pitch” is the fraction of jointly-voiced frames within 50 cents, “onset” is the onset-time F1, and “cents” is the median absolute pitch error over jointly-voiced frames. Higher is better except for cents.

Take	explains	pitch	onset	cents
mary_1	0.59	0.70	0.49	40
repeated_1	0.75	0.92	0.57	20
scale_1	0.54	0.73	0.35	20
twinkle_1	0.61	0.84	0.39	20
mean	0.62			

Pitch agreement is high (between 0.70 and 0.92), which says the resynthesised melody tracks the pitch the person actually hummed and that no gross octave slip occurred. Onset agreement is lower (between 0.35 and 0.57), largely because the onset detector fires on more places in the raw hum than there are notes, so the number understates true agreement. Crucially, because the score compares each transcription with *its own* audio and never with an intended melody, an imprecise hum cannot count against the system: a wrong note the pipeline captures faithfully still agrees. That is what lets these numbers come from a hummer who is not a trained singer. The takes are few (four), so they are read as a sanity signal, not a trend.

One take is worth a separate note. On the Mary-Had-a-Little-Lamb take the pipeline *over-split* the melody (it returned roughly twice as many notes as intended). That is the failure the synthetic *hard* profile could not reproduce (Section 4.5.1), so real voice evidently carries cues, or capture noise, that the synthesiser does not. That take was also quiet and of uncertain tempo, so it is a single weak data point rather than a measured rate; but it is a concrete reason not to over-read the synthetic-only finding that this pipeline does not over-split, and another argument for real ground truth.

What this proves: an unsupervised, real-voice sanity signal that needs no label and that catches gross pitch, onset, and segmentation mismatch, including octave slips. **What it does not prove:** it is a sanity signal, not an accuracy number. A self-consistent error, such as a constant tuning offset within 50 cents, or any error the resynthesis reproduces, leaves the two contours agreeing and is not caught.

4.5.4 Evidence 4: calibrating the synthetic realism from real takes

The fourth kind of evidence closes the loop back to the first. The diagnostic error-rates of Evidence 1 are only as honest as the realism profiles they are measured at, so those profiles are checked against real voice. For each real take the tracker is run and the quantities the realism knobs model are measured: vibrato depth and rate, pitch drift, timing jitter against the metronome, the depth of the consonant energy dips, and amplitude shimmer. Table 9 gives the medians over the four real takes, next to the value the *realistic* profile assumes.

Table 9: Realism quantities measured from the four real takes (medians), next to the value the realistic synthetic profile assumes. The timing-jitter row is the exception: the realistic pitch profile itself keeps timing clean, so the 30 ms shown is the jitter the rhythm test injects (Section 4.4), which is what the measured value is compared against.

Quantity	Measured (median)	Realistic profile
Vibrato depth (cents)	26	55
Vibrato rate (Hz)	5.2	5.3
Pitch drift (cents)	18	18
Timing jitter (ms)	131	30
Closure depth (dB)	5	16
Shimmer (dB)	1.1	—

The vibrato rate and the pitch drift the synthesiser assumes are close to what real humming shows: the rate matches almost exactly (5.2 against 5.3 Hz), and the drift lands on the assumed value (18 cents in both). The assumed vibrato depth is wider than measured (55 against 26 cents), so the realistic profile is, if anything, harder than a real hum on pitch. The measured timing jitter is much larger than the value used in the rhythm test (131 against 30 ms), which says real humming is far looser in time than the rhythm test assumes and is a reason the tempo work of Section 4.7 matters.

What this proves: the synthetic realism is grounded in measured numbers rather than guessed, and two of the central knobs (vibrato rate and drift) match real voice well. **What it does not prove:** four takes is a small sample, and the measurements are estimates from the tracker, not exact.

4.6 Interpreting the scores

The evidence must be read together and read carefully. A saturated note F1 of 1.000 does not mean the system transcribes real hums correctly; it means the synthetic note-detection benchmark can no longer tell a good segmenter from a bad one. The four kinds of evidence above were built precisely because of that. The diagnostic error-rates (Evidence 1) restore a benchmark that discriminates and localise the dominant expressive-input failure to merged same-pitch repeats; the metamorphic properties (Evidence 2) give correctness relations that need no performer; the reconstruction round-trip (Evidence 3) gives a first, unsupervised real-voice signal; and the realism calibration (Evidence 4) keeps the synthetic profiles honest. None of these is a real-world accuracy number, because that still requires real ground truth, but together they are a far more useful picture than one number at ceiling.

The rhythm evaluation once localised the dominant real-world failure to a single cause: sensitivity to tempo error. That diagnosis drove the tempo-refinement stage, which now absorbs a moderate tempo error and, with it, most of the “wrong rhythm” behaviour that a small tempo mismatch used to cause. What remains open is the tempo detector’s behaviour on real humming that drifts beyond the refinement band (Section 4.7), and, more fundamentally, the absence of any real ground truth to measure absolute accuracy against.

4.7 The tempo-detection weakness

The rhythm result above shows that a moderate tempo error is now absorbed by the refinement stage in the quantiser. That stage spans only a bounded band (about $\pm 25\%$ of the stated tempo), however, so a gross error from the system’s own tempo detector still escapes it, and it is worth stating why that detector is fragile.

As described in Section 3.9, the detector returns a single global tempo: it takes the autocorrelation of the onset-strength envelope over the whole clip, weights it by a prior centred near a comfortable humming tempo, and returns the dominant period. This has three consequences on real input. First, it assumes a constant tempo; a hum that speeds up or slows down has onset energy at more than one period, and the autocorrelation returns a blurry compromise rather than either true tempo. Second, the prior actively biases the estimate toward the centre of its range, so a genuinely slow hum is pulled upward. Third, it relies on clean, evenly spaced onsets, which real legato or noisy humming does not always provide. In informal testing, a hum of a roughly 76 beat-per-minute melody that included a faster passage was reported as about 101, a value suspiciously close to the prior, which is the signature of the prior dominating a weak autocorrelation. An error of that size (about $+33\%$) falls outside the refinement band and so is not recovered, which is exactly the case where the detector’s fragility still bites. This weakness is the leading item of future work in Section 5.3.

5 Limitations and Future Work

5.1 Evaluation limitations

The main limitation is the one stated in Section 4.2: no labelled corpus of real hummed melodies exists, so no figure in this report is a real-world accuracy number. The aggregate note and rhythm F1 both saturated during the project. The response was to widen the evaluation (Section 4.5) rather than to keep reporting one number at ceiling: the diagnostic error-rates over the diversified corpus discriminate again, the metamorphic properties and the reconstruction round-trip add signals that need no precise performer, and the realism calibration keeps the synthetic profiles honest. What none of these gives is absolute accuracy on real voice, which still needs real ground truth. A second finding narrows the earlier plan: the over-splitting failure was expected to appear once shimmer, breath, and creak were added to the synthesiser, but direct measurement showed this pipeline does not over-split; its smoothing and consolidation defences hold, and hard input degrades by merging or dropping notes instead. So the open evaluation gap is not over-split synthesis but real ground truth.

The recommended remedy is not a larger synthetic set but a small set of real, labelled hums. The obstacle is ground truth, since a person cannot label the pitch of their own hum by ear. The plan, refined during this project, inverts the labelling. Rather than hum first and label afterward, the label is fixed before the recording: the user hums a melody they know, to a click at a known tempo, and the rhythm ground truth comes from the known tune and the click grid. The pitch of each note is confirmed with an external pitch-finder application used as an assistive aid, with the user verifying each note against the tune they knowingly hummed, rather than trusting the external tool as an oracle. This is

important because an external pitch finder is itself another pitch detector; if it is trusted blindly and it happens to share an error with the pipeline, the evaluation would score that shared error as correct. Keeping the user in the loop, and drawing the rhythm truth from the click rather than from the tool, avoids that circularity. The resulting verified recordings, with their known notes and tempo, would give the project its first real ground truth.

5.2 System deficiencies

Several deficiencies of the current system are known and tracked. Table 10 lists them together with the planned remedy for each.

Table 10: Known deficiencies of the current system and planned work.

Area	Description and planned remedy
Tempo detection fragility	The “find my tempo” estimator returns a single global tempo biased by a prior, and is unreliable on real, variable-tempo, slow, or legato humming (Section 4.7); the quantiser’s tempo-refinement stage absorbs a moderate error but not a gross one. Planned remedy: flatten or remove the prior, and build the onset envelope from the pipeline’s own voicing onsets rather than raw spectral flux.
Merged same-pitch repeats	On expressive input the segmenter merges roughly four in ten of the same-pitch repeats it should keep apart (repeat recall about 0.61, Section 4.5.1). This, not over-splitting, is the dominant expressive-input failure. Planned remedy: strengthen the same-pitch onset cue and re-measure repeat recall as the target.
Over-split does not occur, but was expected	The over-split failure (one note returned as slivers) was expected to appear once shimmer, breath, and creak were added to the synthesiser. Measurement showed this pipeline does not over-split; it merges or drops notes instead (Section 4.5.1). No remedy needed; the expectation is corrected.
No real ground truth	No figure is a real-world accuracy number (Section 4.2); the four real takes feed only the round-trip and calibration, which are performer-tolerant, not an accuracy score. Planned remedy: the hum-to-a-known-score capture flow of Section 5.1.

Continued on the next page.

Area	Description and planned remedy
Aggregate F1 saturated (addressed)	The note-detection and rhythm F1 are at ceiling on the fixtures and no longer discriminate. Addressed by the diagnostic error-rates, metamorphic properties, round-trip, and realism calibration of Section 4.5; the remaining gap is real ground truth.
Monophonic only	The transcription pipeline assumes a single voice, one note at a time. Polyphonic or chordal input is out of scope; the Pitch Finder tab uses a separate chroma-based method for full songs.
Optional backend availability	The most precise backends (PESTO, FCNF0++) are optional installs with heavier dependency trees, and FCNF0++ downloads weights on first use. The pipeline falls back to pyin when they are absent, at a precision cost.
Fixed metre assumption	Quantisation assumes a simple metre and a known, steady tempo. Time-signature changes and expressive rubato are not modelled.

5.3 Future directions

Three directions are recorded for the work after this report, in priority order.

- **(A) Tempo robustness (leading candidate).** With the quantiser’s tempo-refinement stage now absorbing moderate errors, the tempo detector is the largest remaining lever on real-world rhythm accuracy for the gross errors that fall outside the refinement band (Section 4.7), and it is directly testable because the fixtures have known tempos. Two cheap, high-impact changes: flatten or remove the tempo prior so slow tempos are not pulled upward, and build the onset envelope from the pipeline’s own consonant onsets rather than from raw spectral flux.
- **(B) Real-hum ground truth.** The ultimate unlock, blocked only on the labelling method, which Section 5.1 resolves. Four real takes already feed the reconstruction round-trip and the realism calibration (Sections 4.5.3 and 4.5.4); a larger labelled set, captured by the same method, would turn those from a sanity signal into a firmer one and let the project measure real-world accuracy for the first time.
- **(C) The merged-repeat cue.** With over-splitting shown not to occur, the measured weakness is the opposite one: merged same-pitch repeats on expressive input (Section 4.5.1). Strengthening the same-pitch onset cue, with repeat recall as the target metric, is the concrete next step on segmentation.

5.4 Towards production readiness

HumJob is a research and academic project, not a production service. It runs locally, which is a deliberate design choice, but a wider deployment would additionally require packaging the optional neural backends so they install reliably, a broader automated test suite over real recordings, and error handling for the many ways a real microphone recording can be unusable (clipping, background noise, too quiet). None of these is conceptually hard, but together they are a meaningful body of work beyond the scope of this course project.

6 Conclusion

This project set out to transcribe a hummed melody into notes, key, and chords, by accepting two small constraints on how the user hums and building the whole pipeline around them. The transcriber that resulted is a local-first pipeline of pure functions over three data types, with five interchangeable pitch backends, a grid-aware segmenter and consolidator, and a spectral octave-correction stage. Around that core the project grew into a local-first toolkit for the singing voice, delivered as a web application of seven tabs: the transcriber and its interactive editor, a pitch finder, a transposer, a real-time monitor and vocal trainer, a sing-along trainer, an ear trainer, and a practice hub. All of it runs on the user's own machine; the only deliberate exception is a small family of five opt-in, text-only AI features (coaching, a practice plan, natural-language editing, whole-melody reorganisation, and reharmonisation), each of which sends a compact summary and never any audio.

With respect to the research questions, the honest verdicts are mixed. **RQ1** is supported by construction and by the evaluation: the two interface constraints do turn the hard parts into manageable ones, and the grid-aware stages that exploit them are what carried the realistic note F1 from 0.799 to 1.000. **RQ2** is supported: the diagnosed failures were segmentation and quantisation failures over an essentially correct pitch contour, not pitch-model failures. The clearest case is the octave error, which was proven to be a Viterbi decoding artifact rather than a fault in the audio, and was repaired by a spectral test rather than by changing the model. **RQ3** produced the most important lesson of the project. The aggregate note and rhythm F1 the system was tuned against both saturated at 1.000 while the system is still not reliable on real hums, because a synthetic hum is more regular than a real one and no labelled real corpus exists to measure against. Rather than keep reporting a number at ceiling, the evaluation was rebuilt around four kinds of evidence that need no precise singer (Section 4.5). These discriminate again and, between them, say something concrete: the dominant expressive-input failure is merged same-pitch repeats (not, as expected, over-splitting, which this pipeline does not do); the transcriber obeys a set of correctness invariances that also hold on real audio; and a real hum's transcription can be checked against its own audio as an unsupervised sanity signal. What none of them gives is absolute real-world accuracy, which still requires real ground truth.

The main deliverable is therefore not a headline accuracy number. It is a working, well-structured toolkit whose transcription core is paired with an honest evaluation that says clearly where that core stands: strong on the aggregate synthetic benchmark, discriminated again by richer diagnostics, and not yet validated on real input. Four real takes

already anchor the round-trip and realism-calibration signals; the most valuable next step is to grow that into a larger, trustworthy set of real, labelled hums, so that the system’s real-world accuracy can be measured directly rather than inferred from a synthetic proxy.

References

- [1] M. Mauch and S. Dixon. *pYIN: A fundamental frequency estimator using probabilistic threshold distributions*. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2014.
- [2] J. W. Kim, J. Salamon, P. Li, and J. P. Bello. *CREPE: A Convolutional Representation for Pitch Estimation*. ICASSP, 2018. arXiv:1802.06182. Implementation: torchcrepe, <https://github.com/maxrmorrison/torchcrepe>.
- [3] A. Riou, S. Lattner, G. Hadjeres, and G. Peeters. *PESTO: Pitch Estimation with Self-supervised Transposition-equivariant Objective*. International Society for Music Information Retrieval Conference (ISMIR), 2023. <https://github.com/SonyCSLParis/pesto>.
- [4] M. Morrison, C. Hsieh, N. Pruyne, and B. Pardo. *Cross-domain Neural Pitch and Periodicity Estimation*. 2023. arXiv:2301.12258. Implementation: penn, <https://github.com/interactiveaudiolab/penn>.
- [5] R. M. Bittner, J. J. Bosch, D. Rubinstein, G. Meseguer-Brocal, and S. Ewert. *A Lightweight Instrument-Agnostic Model for Polyphonic Note Transcription and Multipitch Estimation*. ICASSP, 2022. <https://github.com/spotify/basic-pitch>.
- [6] C. L. Krumhansl. *Cognitive Foundations of Musical Pitch*. Oxford University Press, 1990. (Krumhansl-Kessler / Krumhansl-Schmuckler probe-tone key profiles.)
- [7] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto. *librosa: Audio and Music Signal Analysis in Python*. Proceedings of the 14th Python in Science Conference (SciPy), 2015. <https://librosa.org/>.
- [8] M. S. Cuthbert and C. Ariza. *music21: A Toolkit for Computer-Aided Musicology and Symbolic Music Data*. ISMIR, 2010. <https://web.mit.edu/music21/>.
- [9] L. Pugin, R. Zitellini, and P. Roland. *Verovio: A library for Engraving MEI Music Notation into SVG*. ISMIR, 2014. <https://www.verovio.org/>.
- [10] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, and D. P. W. Ellis. *mir_eval: A Transparent Implementation of Common MIR Metrics*. ISMIR, 2014. https://github.com/craffel/mir_eval.
- [11] Hollien, Dew, and Philips. *Phonational Frequency Ranges of Adults*. Journal of Speech and Hearing Research, 14:755, 1971. (Population norm for the vocal range finder: means of 38 semitones for males and 37 for females; the standard deviation used is a stated assumption, not from this study.)

Appendix A: System demonstration

This appendix shows the running web application from a user’s perspective. Each subsection corresponds to one tab. The screenshots are captured against a local instance of the stack; small visual details depend on the user’s browser and operating system.

A.1 Hub

The Hub is the home tab: a read-only practice dashboard that folds the history the other tabs record locally into offline trend lines and a compact progress report. Its Coach card adds the opt-in “Get a practice plan” button.

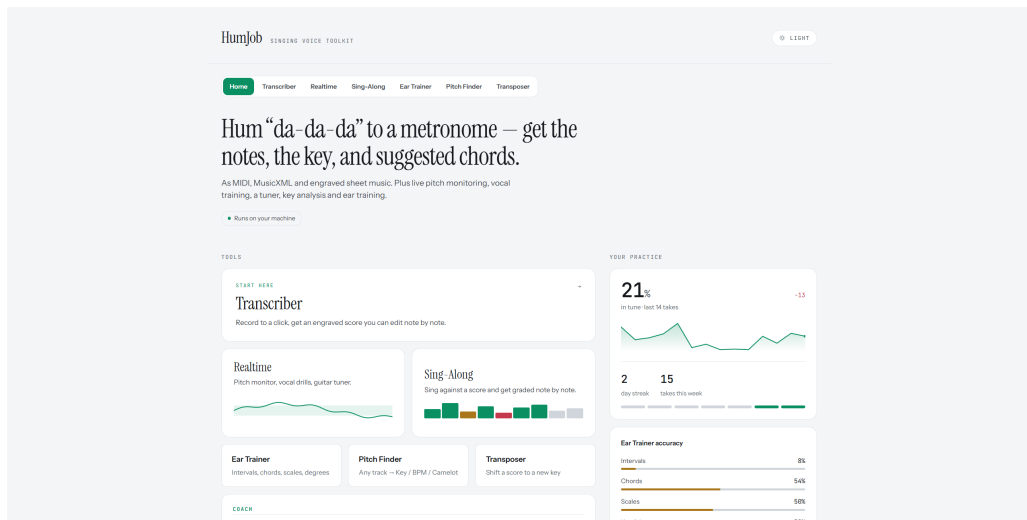


Figure 4: Hub practice dashboard.

A.2 Transcriber: Auto mode

The Transcriber tab records or uploads a hum and returns the automatic draft: the engraved sheet music, the detected key, and the suggested chords, with downloads for MIDI and MusicXML. The chords can be reworked in place: clicking a chord offers offline alternatives, and a “Reharmonise” row asks the opt-in language model for a new progression in a requested style.

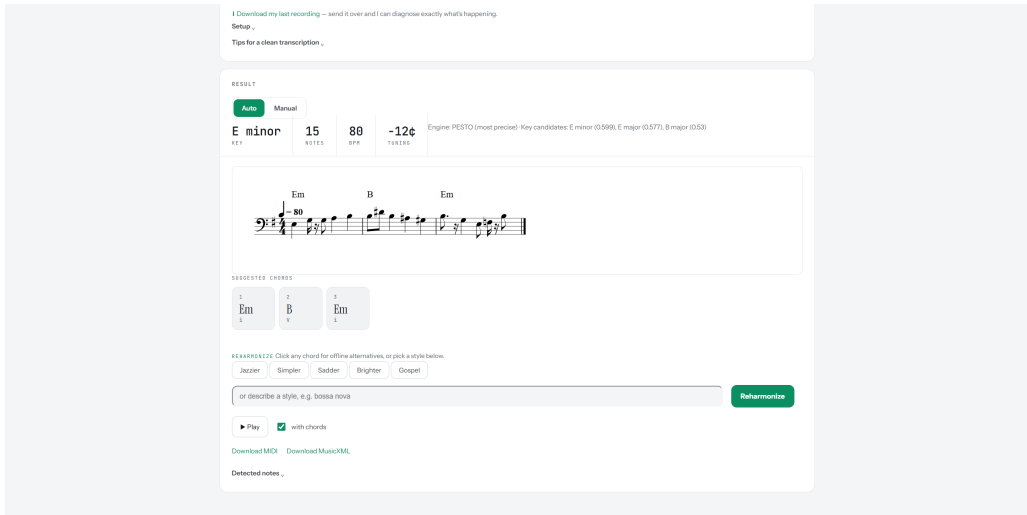


Figure 5: Transcriber: Auto mode.

A.3 Transcriber: Manual mode

Manual mode is the in-browser staff editor for correcting the draft. The reference pitch strip shows the hummed contour against the chosen notes, and the editing toolbar offers pitch, duration, merge, split, delete, and insert with undo and redo. An “Ask” box also accepts plain-language edits, handled by an offline grammar for common phrasings and otherwise by the opt-in language model.

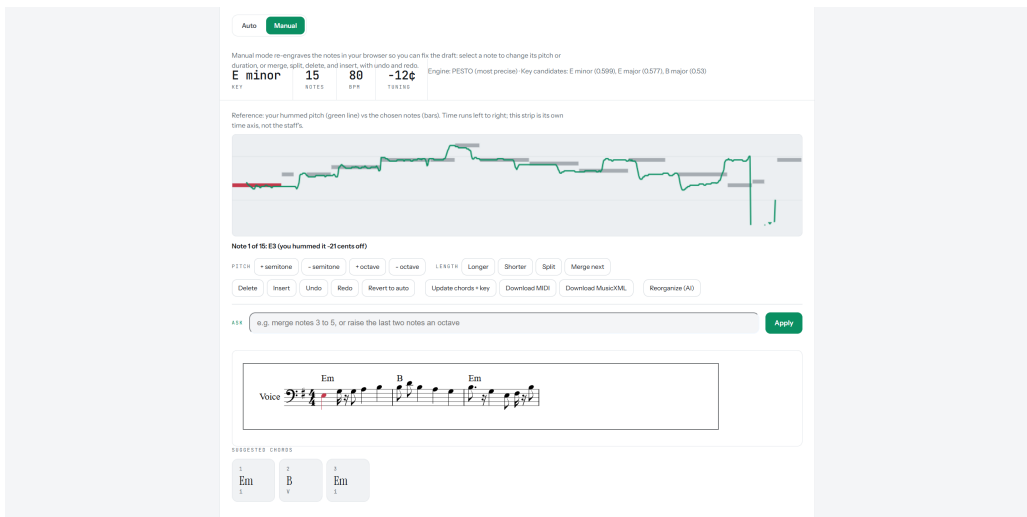


Figure 6: Transcriber: Manual mode.

A.4 Pitch Finder

The Pitch Finder tab analyses an uploaded clip for key, tempo, and a Camelot code, using the chroma-based method that works on polyphonic audio.

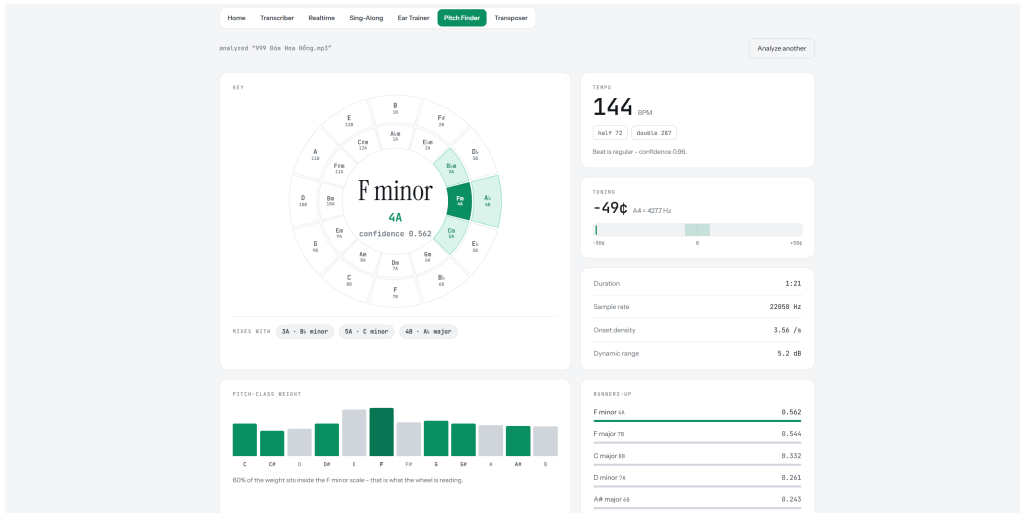


Figure 7: Pitch Finder.

A.5 Realtime and vocal trainer

The Realtime tab is a client-side pitch monitor and guitar tuner with vocal-training drills: a target-note lane, a match game, a scale trainer, vibrato analysis, a vocal range finder, and a per-session progress panel.

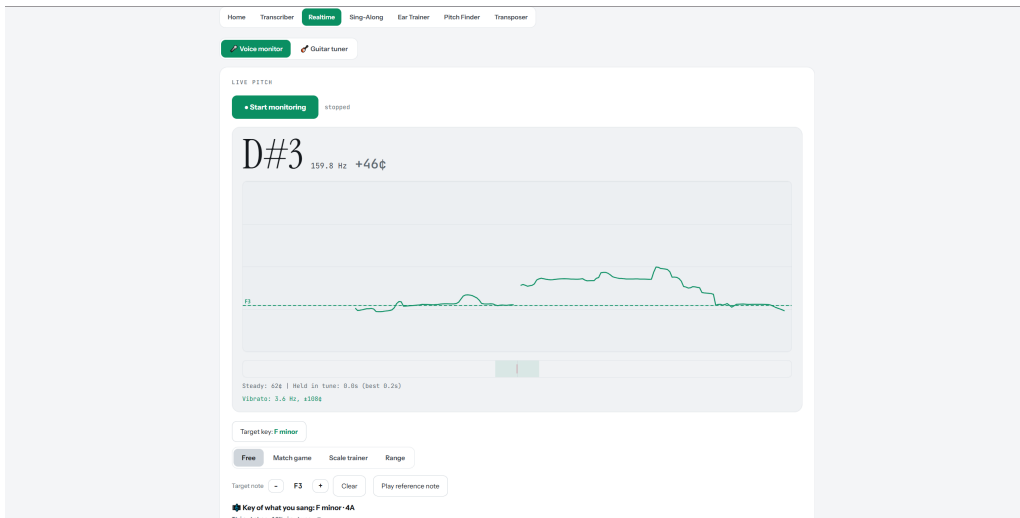


Figure 8: Realtime voice monitor and vocal trainer.

A.6 Transposer

The Transposer tab shifts a score to a new key. File mode transposes an uploaded MIDI or MusicXML file server-side and offers Camelot-compatible key presets; hum mode transposes the last transcription client-side.

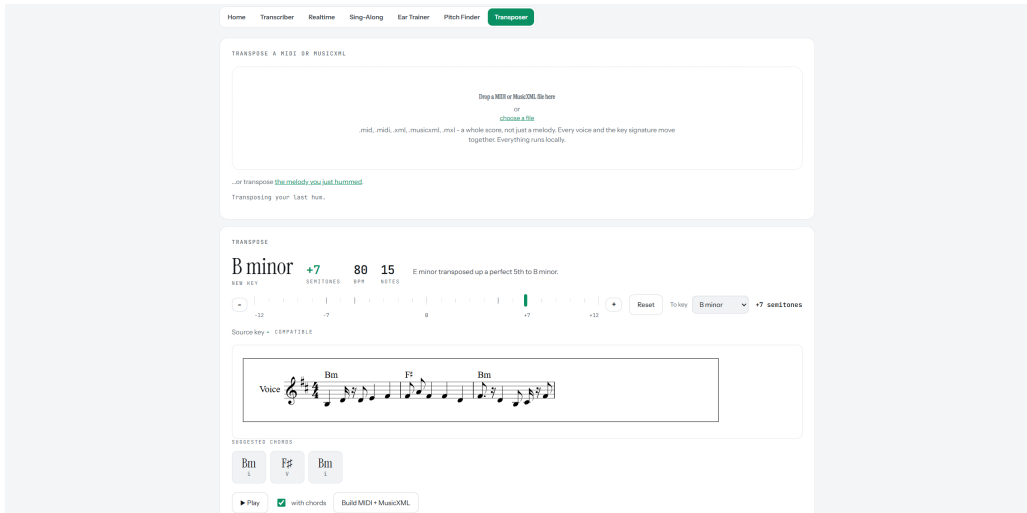


Figure 9: Transposer: hum mode.

A.7 Sing-Along and AI coaching

The Sing-Along tab plays an uploaded score as a guide, tracks the voice live, and scores each note. After a take it shows a deterministic analysis of the performance offline, and, on request, sends a numeric summary to an external language model for spoken-language coaching (one of the opt-in, non-local features described in Section 3.8).

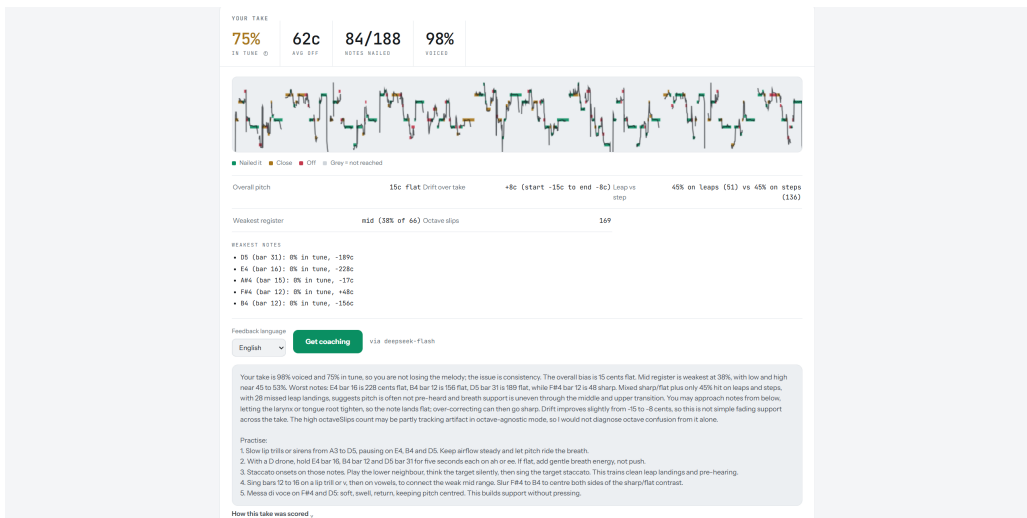


Figure 10: Sing-Along and AI coaching.

A.8 Ear Trainer

The Ear Trainer tab plays a note group and asks the user to identify it by tapping a multiple-choice button, with sub-modes for intervals, chord qualities, scales and modes, and scale degrees, at three difficulty levels. It is fully client-side, with no microphone and no server.

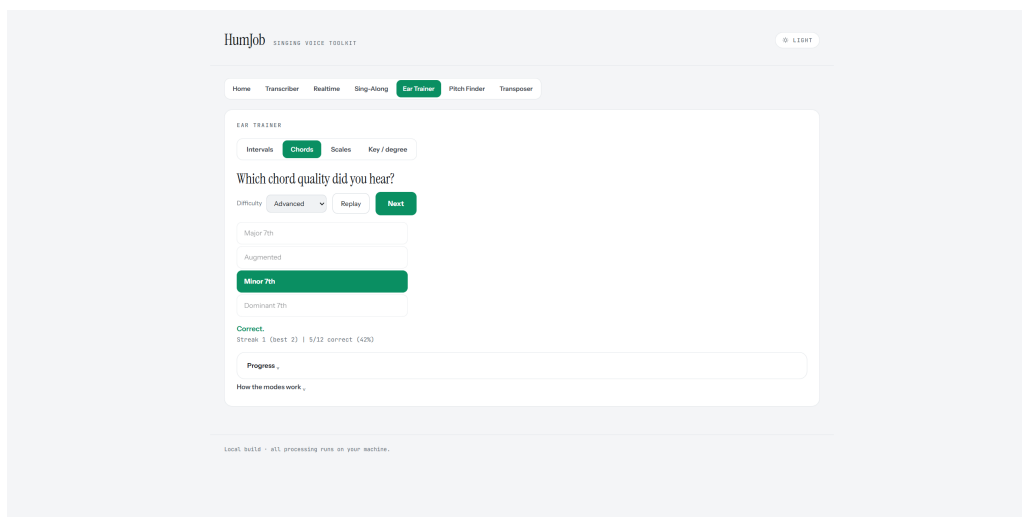


Figure 11: Ear Trainer.